

Lab 2B: Build, Block, and Verify Hooks

Duration: 30 min

After: Hooks lecture (Day 2, Block 2)

Prerequisites: The `business-dashboard` project; Node/npm available; Git Bash or WSL available; Prettier already installed as a pinned project dev dependency

Objective

Choose from the current **31 hook events**, wire a deterministic `PostToolUse` command hook, build a secret blocker that immediately denies unsafe writes with `exit 2`, watch Claude react to the block feedback, scope it with a **matcher**, and verify the finished guardrail with a repeatable test matrix.

Deliverable (toolkit chain 6 of 12)

Toolkit artifact #3: registered and tested hooks (`.claude/hooks/` + `.claude/settings.json`), including a repeatable hook test script.

***START MARKER:** In `business-dashboard`, use a Git Bash or WSL terminal and confirm `bash --version` succeeds. If Bash is missing, install Git for Windows with Git Bash or enable WSL before continuing. Confirm `npm prettier --version` succeeds without an install prompt; if it does not, ask the instructor to restore the pinned project dependency rather than allowing an in-class download. After adding a hook or changing its registration, open a new Claude Code tab/session in VS Code before testing it. Once the new session is open in the same project, close the old tab. `/clear` does not reload hooks.*

Exercise 0: Choose the Event and Handler (3 min)

Hooks are external handlers that run on lifecycle events. The current reference contains **31 events**; learn the high-value pairs and use the Hooks Events Reference for the full map. There are **5 handler types**: `command`, `http`, `mcp_tool`, `prompt`, and `agent`. Today you build deterministic `command` handlers.

Group	High-value events
Tools and permissions	PreToolUse , PostToolUse , <code>PostToolUseFailure</code> , <code>PermissionRequest</code> , <code>PermissionDenied</code>
Session and prompt	<code>SessionStart</code> , <code>SessionEnd</code> , <code>UserPromptSubmit</code> , <code>Stop</code> , <code>Notification</code>
Agents and workspace	<code>SubagentStart</code> , <code>SubagentStop</code> , <code>PreCompact</code> , <code>PostCompact</code> , <code>WorktreeCreate</code>

Warm-up: format after an edit -> **PostToolUse**. Stop a secret before it lands -> **PreToolUse**.

Exercise 1: PostToolUse Format Hook (7 min)

Create the formatting hook (advisory by nature - it fixes, never blocks):

Create `.claude/hooks/post-edit-format.sh` that:

- Reads JSON from stdin
- Extracts `.tool_input.file_path`
- Exits 0 when there is no path
- Processes only `.js/.ts/.jsx/.tsx` files
- Runs `npx prettier --write` when `npx` is available
- Is executable

Merge a `PostToolUse` hook into `.claude/settings.json`:

```
{
  "hooks": {
    "PostToolUse": [
      { "matcher": "Write|Edit",
        "hooks": [
          { "type": "command", "command": "bash",
            "args": ["${CLAUDE_PROJECT_DIR}/.claude/hooks/post-edit-format.sh"] }
        ] }
      ]
    }
  }
```

Do not overwrite existing permissions.

Matchers are exact and case-sensitive: `Write`, `Edit`, and `Bash`.

Load the hook before testing: open a new Claude Code tab/session in VS Code, return to the same `business-dashboard` project, and close the old tab.

If `server.js` already contains a `formatProbe` from an earlier attempt, delete it first. Then send Claude this exact prompt:

Use the Edit tool to add this exact line to `server.js`. Keep it as one unformatted line so the `PostToolUse` hook can format it:

```
const formatProbe={hook:"PostToolUse",ready:true};
```

Do not paste the line into VS Code yourself. The hook runs after Claude uses its `Write` or `Edit` tool; manual editor changes do not trigger it.

Re-open `server.js` and compare the result.

PASS - the hook ran: Prettier adds standard spacing, producing a line similar to:

```
const formatProbe = { hook: "PostToolUse", ready: true };
```

Exact quote style may vary. The visible proof is that spaces have been added around `=` and inside the object.

FAIL - investigate before continuing: the file still contains the original compact form:

```
const formatProbe={hook:"PostToolUse",ready:true};
```

If it fails, confirm Claude actually used `Edit`, confirm the new session loaded `.claude/settings.json`, and inspect the hook output for an error. Remove `formatProbe` after the hook passes.

Do not test Prettier with spacing inside a comment. Formatters normally preserve comment text, so an

unchanged comment does not prove that the hook failed.

Exercise 2: Build and Prove a Blocking Secret Detector (8 min)

This control should stop a hardcoded secret before it reaches disk and give Claude an actionable reason.

```
Create or REPLACE .claude/hooks/pre-write-check.sh. If it already exists from an earlier lab
run,
overwrite it. This version must block immediately:
- Read hook JSON from stdin
- Extract .tool_input.content
- Exit 0 when the content is empty or safe
- Detect content matching a hardcoded API key, secret, password, token, or sk- credential
- On detection, print this reason to stderr:
Hardcoded secret detected - use an environment variable instead
- Exit 2
- Be executable
```

Register it in .claude/settings.json as a PreToolUse command hook with matcher "Write" using exec-form arguments so the project path is handled reliably on Windows:

```
{ "type": "command", "command": "bash",
  "args": ["${CLAUDE_PROJECT_DIR}/.claude/hooks/pre-write-check.sh"] }
```

Replace any older PreToolUse registration for this script; do not append a duplicate hook.

Direct checkpoint before the live test: in the VS Code Git Bash or WSL terminal, run these exact commands:

```
safe='{ "tool_input": { "content": "const apiKey = process.env.API_KEY" } }'
bad='{ "tool_input": { "content": "const API_KEY = \"FAKE_TEST_SECRET_12345\"" } }'
printf '%s' "$safe" | bash .claude/hooks/pre-write-check.sh; echo $?
printf '%s' "$bad" | bash .claude/hooks/pre-write-check.sh; echo $?
```

The safe command must print exit **0**. The unsafe command must print exit **2** and the exact stderr reason.

Do not continue unless both direct checks pass.

Load the hook: open a new Claude Code tab/session in VS Code, return to the same project, and close the old tab. Ensure `test-secret.js` does not already exist, then run:

```
Create test-secret.js containing exactly: const API_KEY = "FAKE_TEST_SECRET_12345"
```

Expected output markers: the `Write` is blocked before execution, the actionable reason is fed back to Claude, and `test-secret.js` does **not** contain the hardcoded secret. Claude may retry with `process.env.API_KEY` or explain why it cannot complete the original request. Either reaction proves that the blocker steered the agent.

Confirm the safe path still works:

```
Create test-safe.js containing: const apiKey = process.env.API_KEY
```

Expected output marker: the safe file is written. Delete `test-secret.js` if Claude created a safe replacement, and delete `test-safe.js` after verification.

*Exit 0 allows the action. On **PreToolUse**, exit 2 blocks before execution and stderr carries feedback to Claude. A **PostToolUse** hook cannot undo an action that already completed.*

Exercise 3: Scope the Matcher (3 min)

Your blocker's matcher is "Write" - find the gap:

```
Open server.js and use Edit to insert this line near the top: const API_KEY =
"FAKE_TEST_SECRET_12345"
```

Expected output marker: the secret lands - the hook never fired, because **Edit** doesn't match "Write". Remove the line, then close the gap:

```
Change the pre-write-check matcher in .claude/settings.json from "Write" to "Write|Edit",
and make the script also check .tool_input.new_string when .tool_input.content is empty.
```

Open a new Claude Code tab/session, return to the same project, and close the old tab. Retry the Edit prompt. **Expected output marker:** blocked. Lesson: matchers scope rollout - start narrow, widen deliberately, and test the tools you *didn't* match.

Exercise 4: Build a Repeatable Hook Test Matrix (4 min)

Interactive prompts prove the hook works once. A small test script makes the guardrail safe to change later.

```
Create .claude/hooks/test-pre-write-check.sh that runs pre-write-check.sh with three JSON
payloads:
```

1. Write with safe content: const apiKey = process.env.API_KEY - expect exit 0
2. Write with secret content: const API_KEY = "FAKE_TEST_SECRET_12345" - expect exit 2
3. Edit with secret new_string: const API_KEY = "FAKE_TEST_SECRET_12345" - expect exit 2

```
Print PASS or FAIL for each case, print a final "3/3 passed" summary when successful,
exit 1 if any expectation fails, and make the script executable. Then run it with Bash.
```

Expected output marker: all three cases print **PASS**, the summary says 3/3 passed, and the test script exits **0**. If a case fails, inspect the exact payload field and exit code before changing the hook.

This is the production lesson: hooks are executable controls, so test both the allowed path and every tool-input path you intend to block.

Exercise 5: Commit the Toolkit (2 min)

```
Stage and commit .claude/hooks/ and .claude/settings.json with message "feat: add tested
enforcement hooks"
```

Optional Challenge: Expand the Security Regression Matrix (8-10 min)

Finish the core lab first. Extend `test-pre-write-check.sh` with these three cases without weakening the existing checks:

1. Write with const password = "FAKE_TEST_PASSWORD_12345" - expect exit 2.

2. Write with `const token = process.env.TOKEN` - expect exit 0.
3. Edit with `new_string` set to `const apiKey = process.env.API_KEY` - expect exit 0.

Run all six cases. If one fails, change the smallest possible part of `pre-write-check.sh`, then rerun the entire matrix rather than only the failed case.

Challenge done when: the script reports 6/6 `passed`, both environment-variable cases remain allowed, and the original unsafe Write and Edit fixtures remain blocked.

The lesson is regression safety: every new detection rule needs both a blocking example and a nearby safe example so improving coverage does not silently increase false positives.

VS Code Acceptance Gate and Process Receipt

Before commit, capture the same supervised workflow used across Day 2:

1. **Orient:** select the target range in the editor and use `Alt+K` or an explicit `@file#line-range`; explain unfamiliar code before editing.
2. **Plan:** review the bounded plan in the Claude Code panel; add an inline comment when the plan misses a constraint.
3. **Approve and implement:** approve only the requested scope and exact permissions.
4. **Inspect diff:** review every changed file in VS Code Source Control and reject unexpected files or authority changes.
5. **Verify:** clear the Problems panel for the touched scope and run the lab's deterministic tests or fixtures in the integrated terminal.
6. **Decide:** record the request, approved plan, changed files, verification evidence, and the human commit/install decision.

A model summary or grader result supports this receipt; it does not replace the diff review.

FINISH MARKER - Success Checklist

- ☐ You can identify the high-value hook event pairs and all 5 handler types
- ☐ `PostToolUse` formatting runs on `Write|Edit`
- ☐ A new Claude Code tab/session was opened after each hook registration change
- ☐ Blocking secret fixture exits 2, is stopped before writing, and Claude receives the actionable reason
- ☐ Safe environment-variable fixture exits 0 and is written successfully
- ☐ Matcher gap found via `Edit`, closed with `Write|Edit`, retested
- ☐ Hook test matrix passes safe `Write`, blocked `Write`, and blocked `Edit` cases
- ☐ Toolkit committed

If Stuck

Problem	Recovery
Hook never fires	Check matcher case, settings JSON, <code>bash --version</code> , and the exec-form <code>args</code> path; then open a new Claude Code tab/session because <code>/clear</code> does not reload hooks
<code>jq</code> unavailable	Ask Claude to implement the hook in Node or Python instead of weakening the check

Problem	Recovery
Hook blocks everything	Pipe safe sample JSON into the script and inspect stdout plus exit status
Claude doesn't react to the block	Make the stderr reason actionable ("use an environment variable instead") - the agent acts on what it can read
Hook test matrix fails	Compare the failing case's payload field and expected exit code with the direct script output before editing the hook
Prettier unavailable	Ask the instructor to restore the pinned project dependency; do not allow <code>npm</code> to download a package during class

Minimum path if behind: complete Exercises 2-3 and prove the final `Write|Edit` PreToolUse blocker on both the exact unsafe fixture and `const apiKey = process.env.API_KEY`. The formatter, repeatable test matrix, and commit can be completed from the checkpoint after the blocking control works.

Debrief Questions

1. Which rules require deterministic hooks rather than model judgment?
2. Why test the script directly with safe and unsafe payloads before testing it through Claude Code?
3. What made Claude self-correct after the block - and what does that imply about writing good deny reasons?
4. Why must a hook test both allowed content and every matched tool-input path?